ALTORO MUTUAL

Web Application Security Assessment

Four verified security findings

Done by: Syeda Juwayriyah Kahkashan

Contact info.

Email: juwayriyahkahkashan9114@gmail.com

Linkedin id: www.linkedin.com/in/syeda-juwayriyah-kahkashan

Target:	Altoro Mutual demo application — **http://testfire.net**
Assessment type:	Black-box, manual, non-destructive security testing
Environment:	Authorized vulnerable training/demo application
Tools:	Web browser, Kali Linux terminal (curl), Swagger/OpenAPI

THE REPORT IS MADE KEEPING IN MIND THE PROVIDED REFERECE AND FORMAT

1. Summary

This assessment examined the Altoro Mutual web application, a deliberately vulnerable sample banking application used for security training. The assessment focused on common web-application weaknesses that could be demonstrated safely through the browser and Kali Linux.

Four findings were selected for the final report: SQL injection in the login mechanism, reflected XSS through the search function, XSS in the feedback function, and missing HTTP security response headers. The strongest directly captured proof is the SQL injection authentication bypass, where the supplied payload resulted in an authenticated session displaying "Hello Admin User."

The two XSS tests were documented with the payloads and affected endpoints in the assessment notes. Published Altoro Mutual testing material also independently identifies the search and feedback locations as XSS-prone. The security-header test was performed from Kali using curl and showed that the tested response did not include Content-Security-Policy, X-Frame-Options, or X-Content-Type-Options.

Key Findings

#	Finding	Severity	Primary evidence
1	SQL Injection — login authentication bypass	High	Before/after login screenshots + payload
2	Reflected XSS — search query	Medium	Payload + affected URL; independent Altoro validation
3	XSS — feedback name field	Medium	Before/after feedback screenshots + payload
4	Missing HTTP security response headers	Low	Kali curl response headers

Conclusion

The assessment demonstrates that insecure input handling and weak browser-side security controls can create significant security exposure in a banking-style application. The most important corrective actions are to use parameterized database queries, apply context-aware output encoding and sanitization, and configure appropriate HTTP security headers.

2. Scope and Methodology

Target: http://testfire.net

The supplied notes recorded the following areas during reconnaissance:

- Search, Feedback, Contact Us, Personal Checking, Personal Other Services, Small Business and Sign In.
- Authenticated areas including My Account, View Account Details, Recent Transactions and Transfer Funds.
- REST API documentation at http://testfire.net/swagger/index.html.

The REST API documentation was inspected, but no authorization weakness was claimed because the /account endpoint explicitly required an Authorization token. No destructive API operation was executed.

Testing was performed manually. Burp Suite was not used. Browser developer tools were unavailable for the header test, so the response headers were checked from Kali Linux using curl.

Evidence and reporting standard

The report distinguishes between findings directly demonstrated in the user's evidence and findings additionally supported by reputable external documentation. Where the student's captured evidence did not show a JavaScript alert, the report does not claim that an alert was personally observed.

FINDING 1

SQL Injection — Login Authentication Bypass

RISK-HIGH

Description

SQL injection occurs when untrusted input is incorporated into a database query in a way that allows the input to change the intended SQL logic. In the Altoro Mutual login test, the username field accepted SQL syntax that altered the authentication condition.

The supplied proof-of-concept used the username payload below and an arbitrary password. The resulting page displayed "Hello Admin User," demonstrating that the login control could be bypassed.

Affected URL and parameter

- URL: http://testfire.net/login.jsp
- Authentication endpoint observed by the application: /doLogin
- Vulnerable input: Username (uid/login username field).

Payload used

```
Username: admin' OR '1'='1
Password: anything
```

Proof of Concept — step-by-step

1. Open the Altoro Mutual login page.
2. Enter the SQL injection payload in the Username field.
3. Enter any value in the Password field.
4. Submit the login form.
5. Observe that the application enters an authenticated area and displays "Hello Admin User."
6. The post-login page also exposes account-related and administration navigation, providing direct evidence that authentication was bypassed.

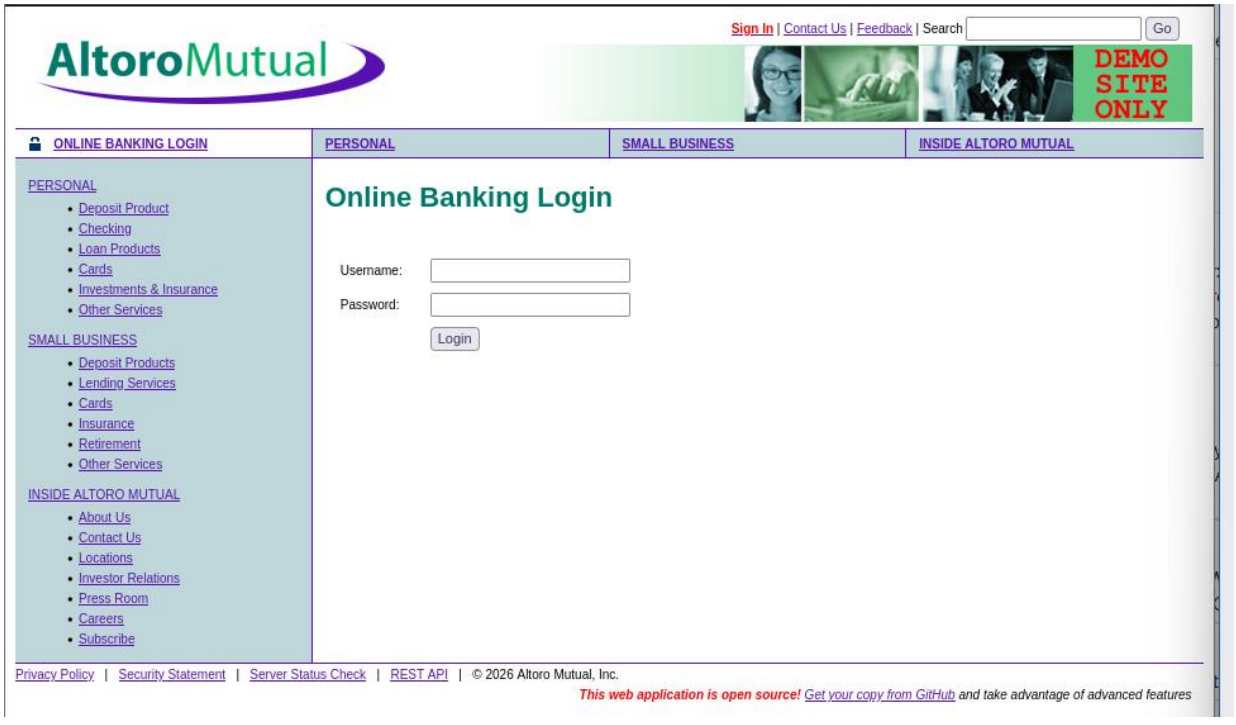


Figure 1. Login page before the SQL injection test.

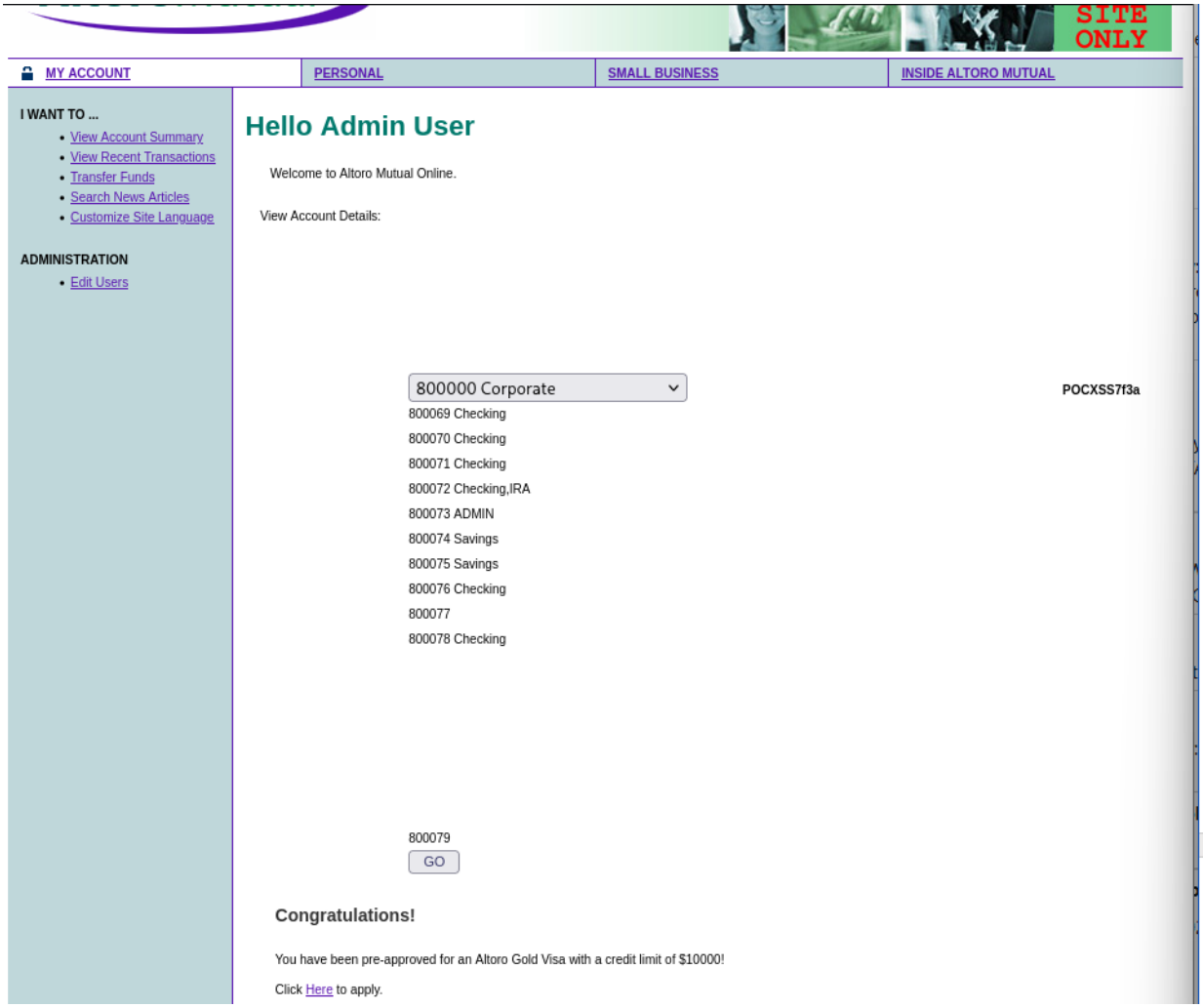


Figure 2. Post-login page showing "Hello Admin User" after the SQL injection test.

IMPACT

- Authentication bypass can allow an unauthorized party to access accounts without a valid password.
- In a banking application, unauthorized account access could expose financial information and customer data.
- If privileged accounts are reachable, the Impact can extend beyond one user's data to administrative functions.
- SQL injection can potentially lead to unauthorized database reads or modifications, depending on the database account privileges.

Remediation / countermeasures

- Replace dynamically constructed SQL statements with prepared statements / parameterized queries.
- Treat all authentication inputs as untrusted data and validate them against expected formats.
- Use a least-privilege database account so the application cannot perform unnecessary database operations.
- Add automated security tests for authentication bypass and SQL injection regressions.

References

OWASP — SQL Injection Prevention Cheat Sheet: OWASP SQL Injection Prevention

OWASP Vulnerable Web Applications Directory — Altoro Mutual: Altoro Mutual (AltoroJ)

FINDING 2

Reflected Cross-Site Scripting — Search Query

RISK-MEDIUM

Description

Reflected Cross-Site Scripting (XSS) occurs when attacker-controlled input is returned in a web response without appropriate context-aware output encoding or sanitization, allowing the browser to interpret the input as active content.

The assessment notes record a JavaScript payload submitted through the Search function and provide the resulting encoded query URL. Independent security testing of Altoro Mutual also identifies /search.jsp and the query parameter as a true-positive XSS location.

Affected URL and parameter

- URL: http://testfire.net/search.jsp
- Parameter: query

Payload and observed request

```
<script>alert('XSS')</script>
http://testfire.net/search.jsp?query=%3Cscript%3Ealert%28%27XSS%27%29%3C%2Fscript%3E
```

Proof of Concept — step-by-step

7. Open the Search function on Altoro Mutual.
8. Enter <script>alert('XSS')</script> as the search query.
9. Submit the search.
10. Observe the resulting search request. The supplied evidence records the URL-encoded payload in the query parameter.
11. For a complete visual PoC, the expected security indicator is execution of the supplied JavaScript in the page context; the student's captured document does not contain an alert screenshot, so this report does not claim that the alert was personally captured.

IMPACT

- A crafted link could cause JavaScript to execute in a victim's browser when the link is opened.
- Depending on browser context and application functionality, XSS can support phishing, unauthorized actions, page manipulation, or access to data available to the page.
- In a banking application, successful XSS can undermine user trust and potentially expose sensitive session or page data.

Remediation / countermeasures

- Perform strict input validation where an allow-list is appropriate.
- Apply context-aware output encoding before placing user-controlled values into HTML, attributes, JavaScript, CSS or URLs.
- Avoid inserting untrusted strings into dangerous browser contexts.
- Deploy a carefully designed Content-Security-Policy as an additional defense-in-depth control.

References

OWASP — Cross Site Scripting Prevention Cheat Sheet: OWASP XSS Prevention

OWASP ZAP — AltoroJ test application results: ZAP AltoroJ documentation

FINDING 3

Cross-Site Scripting — Feedback Function

RISK-MEDIUM

Description

The Feedback function accepts user-controlled text in the feedback form. The assessment notes record a JavaScript payload submitted in the form and the resulting /sendFeedback endpoint.

Independent Altoro Mutual security assessments identify the feedback functionality as an XSS location, including the name field. The captured before/after screenshots in this assessment show the feedback form and the resulting Thank You page after submission.

Affected URL and parameter

- Input page: http://testfire.net/feedback.jsp
- Submission endpoint: http://testfire.net/sendFeedback
- Vulnerable input tested: Your Name / name

Payload used

```
<script>alert('XSS')</script>
```

Proof of Concept — step-by-step

12. Log in to the Altoro Mutual demo account and open the Feedback page.
13. Place the test payload in the Your Name field.
14. Submit the feedback form.
15. Observe that the application accepts the input and returns a Thank You response from the /sendFeedback endpoint.
16. The assessment notes record the payload and submission endpoint. The supplied screenshots document the before and after states.

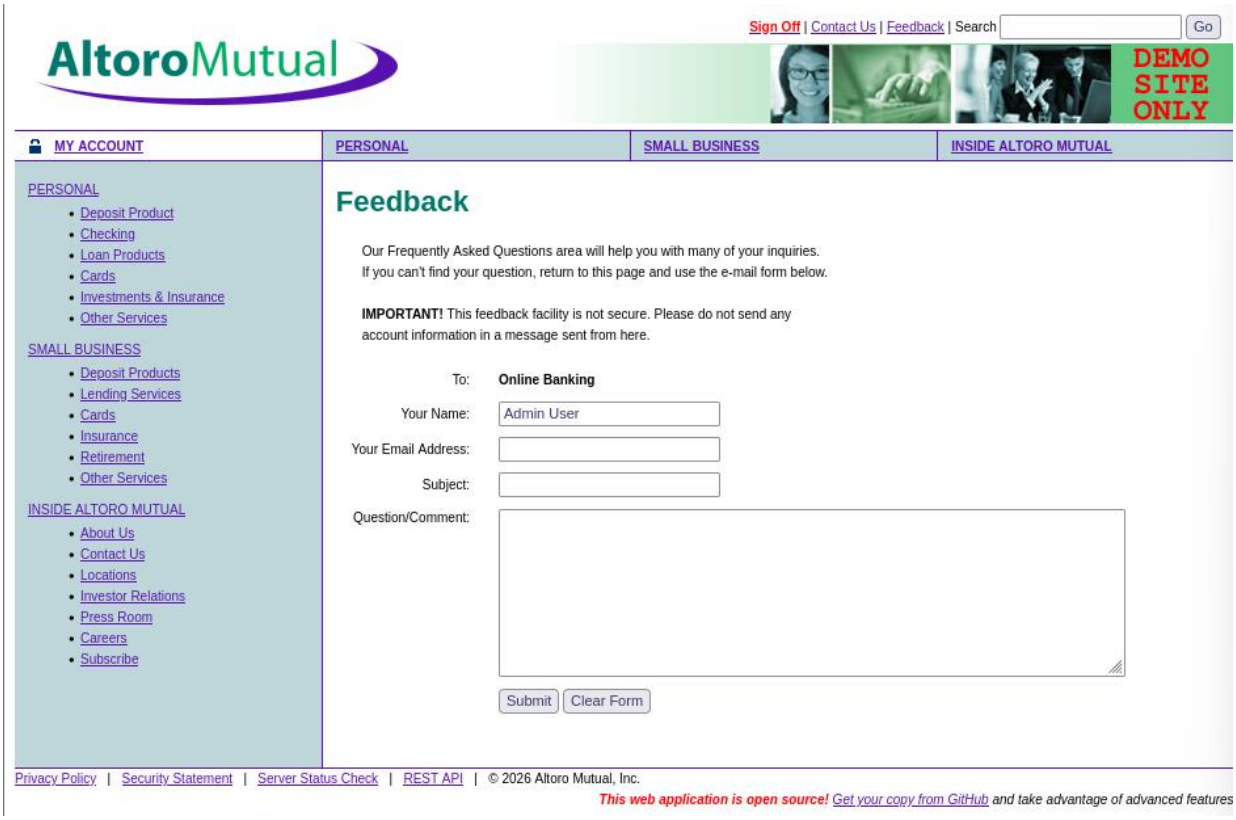


Figure 3. Feedback form before submission.



Figure 4. Thank You response after feedback submission.

IMPACT

- If submitted input is later rendered without proper encoding, JavaScript can execute in another user's browser.
- Potential consequences include phishing, malicious page modification, unauthorized browser actions and exposure of information available to the affected page.

- Because the function is part of a banking-style application, successful client-side injection can reduce user confidence and increase the risk of social-engineering attacks.

Remediation / countermeasures

- Validate the expected content and length of the name field.
- HTML-encode the value whenever it is inserted into an HTML response.
- Sanitize rich text only when rich text is genuinely required; otherwise treat the field as plain text.
- Use a defense-in-depth Content-Security-Policy.

References

OWASP — Cross Site Scripting Prevention Cheat Sheet: OWASP XSS Prevention

Independent Altoro Mutual penetration-test report: Altoro Mutual pentest report

FINDING 4

Missing HTTP Security Response Headers

RISK-LOW

Description

HTTP security response headers provide browser-side security controls and defense in depth. The Kali Linux header check returned a successful HTTP response, but the response did not contain Content-Security-Policy, X-Frame-Options, or X-Content-Type-Options.

These are configuration weaknesses rather than a single stand-alone exploit. Their absence can make other attacks easier or remove browser protections that would otherwise reduce Impact.

Tested URL and command

- URL: http://testfire.net/
- Method: HTTP HEAD request
- Tool: Kali Linux curl

```
curl -I http://testfire.net/
```

Observed response

```
HTTP/1.1 200 OK
Server: Apache-Coyote/1.1
Set-Cookie: JSESSIONID=27FCBDE6C443B38D15603F9A0785D7A2; Path=/; HttpOnly
Content-Type: text/html;charset=ISO-8859-1
Transfer-Encoding: chunked
Date: Wed, 23 Sep 2026 04:48:03 GMT
```

Proof of Concept



Figure 5. Terminal command

Results

Header	Result	Security purpose
Content-Security-Policy	Missing	Restricts script/content sources and provides XSS defense in depth.
X-Frame-Options	Missing	Helps prevent a page from being embedded for clickjacking.
X-Content-Type-Options	Missing	Helps prevent MIME-type sniffing.

The same response also exposes the server banner “Apache-Coyote/1.1”. This is recorded as an informational observation rather than a separate finding in this four-finding report.

IMPACT

- Without CSP, the browser has fewer restrictions available to limit the Impact of script-injection vulnerabilities.
- Without X-Frame-Options (or an equivalent CSP frame-ancestors policy), clickjacking defenses are reduced.
- Without X-Content-Type-Options, browsers may have fewer protections against MIME-type sniffing in applicable responses.
- These weaknesses are generally defense-in-depth issues and should be prioritized alongside the directly exploitable input-validation findings.

Remediation / countermeasures

- Deploy a restrictive, application-specific Content-Security-Policy and test it before enforcing it.
- Use X-Frame-Options where appropriate, or preferably CSP frame-ancestors for modern applications.
- Set X-Content-Type-Options: nosniff on appropriate HTTP responses.
- Review other modern security headers and enforce HTTPS/TLS across the application.

References

OWASP — HTTP Headers Cheat Sheet: OWASP HTTP Security Headers

OWASP — Content Security Policy Cheat Sheet: OWASP CSP

OWASP Secure Headers Project: OWASP Secure Headers

5. Reconnaissance and API Review

The supplied assessment notes recorded several application URLs and the REST API documentation. The Swagger interface exposed login, account, transfer, feedback, admin and logout operations.

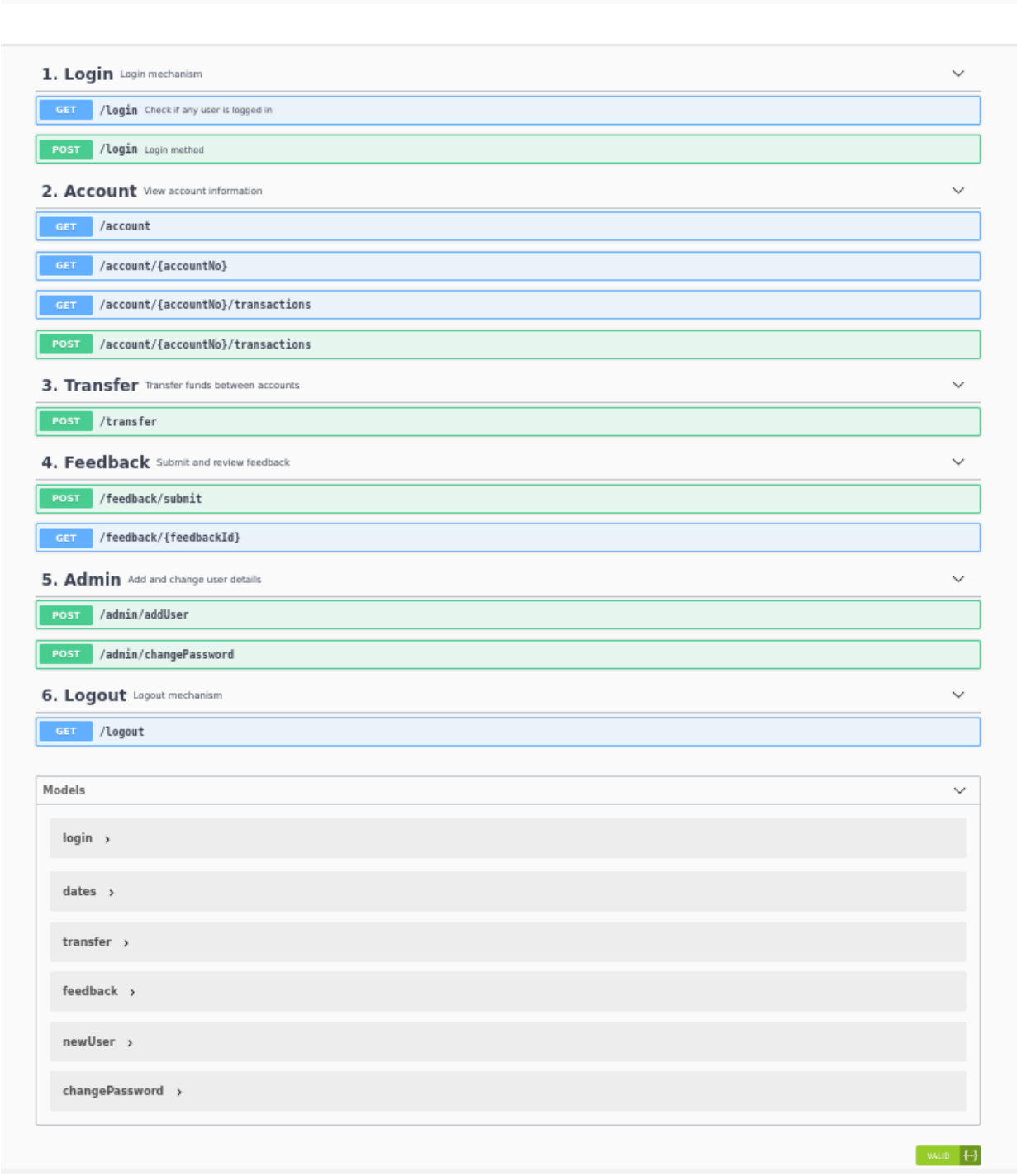


Figure 6. Altoro Mutual REST API documentation (Swagger/OpenAPI).

Authorization observation

The GET /account endpoint explicitly required an Authorization header described as an authorization token provided after successful login. Because access control was enforced at this endpoint during the test, no broken-access-control/IDOR finding was claimed.

Why this matters

A professional assessment should report what the evidence demonstrates rather than forcing a vulnerability classification when the test does not support it.

6. Limitations and Evidence Quality

- Testing was limited to the deliberately vulnerable Altoro Mutual demonstration environment and was non-destructive.
- No database dumping, destructive API operation, password modification, money transfer, or denial-of-service activity was performed.
- Browser developer tools were not used successfully for the header test; curl was used instead.
- The SQL injection authentication bypass has direct before/after screenshots.
- The feedback XSS test has direct before/after page screenshots and the payload/endpoint in the assessment notes.
- The search XSS test has the payload and exact encoded URL in the assessment notes, while an alert screenshot was not captured.
- The missing-header finding is directly supported by the recorded curl response.

7. Prioritized Remediation Plan

The following order is a practical remediation sequence based on the evidence collected in this assessment:

Step	Action	Recommendation
1	Fix SQL injection in authentication	Use parameterized queries/prepared statements and regression-test the login workflow.
2	Fix both XSS injection points	Validate input, encode output according to context, and add defense-in-depth CSP.
3	Harden HTTP response headers	Configure CSP, X-Frame-Options or CSP frame-ancestors, and X-Content-Type-Options.
4	Reduce information disclosure	Review server banners and avoid exposing unnecessary implementation details.
5	Retest	Repeat the same PoCs after remediation and retain before/after evidence.

8. Conclusion

The assessment demonstrates that insecure input handling and weak browser-side security controls can create significant security exposure in a banking-style application. The most important corrective actions are to use parameterized database queries, apply context-aware output encoding and sanitization, and configure appropriate HTTP security headers.

9. References

THE REPORT HAS BEEN MADE KEEPING IN MIND THE PROVIDED REFERECE AND FORMAT.

- OWASP Vulnerable Web Applications Directory — Altoro Mutual (AltoroJ) — https://vwad.owasp.org/app/altoro-mutual-altoroj/
- OWASP SQL Injection Prevention Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

- OWASP Cross Site Scripting Prevention Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- OWASP HTTP Security Response Headers Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/HTTP_Headers_Cheat_Sheet.html
- OWASP Content Security Policy Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html
- OWASP Secure Headers Project — https://owasp.org/projects/secure-headers-project
- OWASP ZAP — AltoroJ test application documentation — https://github.com/zaproxy/zaproxy-website/blob/main/site/content/docs/testapps/altoroj.md
- Altoro Mutual pentest report — independent supporting reference — https://github.com/lksloic/altoro-bank-pentest-report

The different URL explored:

- Search URL recorded: http://testfire.net/search.jsp?query=jh
- Feedback URL recorded: http://testfire.net/feedback.jsp
- Sign-in URL recorded: http://testfire.net/login.jsp
- REST API URL recorded: http://testfire.net/swagger/index.html
- SQLi username: admin' OR '1'='1
- SQLi password: anything
- Search XSS payload: <script>alert('XSS')</script>
- Search XSS encoded URL: http://testfire.net/search.jsp?query=%3Cscript%3Ealert%28%27XSS%27%29%3C%2Fscript%3E
- Feedback XSS payload: <script>alert('XSS')</script>
- Feedback submission endpoint: http://testfire.net/sendFeedback